中国科学院大学
《计算机组成原理(研讨课)》实验报告

姓名 王浚沣 学号 2024K8009909018 专业 计算机科学与技术

实验项目编号 3 实验名称 定制功能型处理器设计

注 1:	撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下 (注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中 "-" 为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
注 2:	使用 `git add` 及 `git commit` 命令将实验报告 `PDF` 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
注 3:	实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

完整实验流程包括设计、开发、调试三个步骤:

- **设计**步骤为在代码编写前的准备阶段, 主要包括模块划分和模块结构设计, 并绘制设计图纸。
- **开发**步骤为在 S-IDE 集成开发环境中进行 Verilog/C 代码编写, 并实现完整功能。
- **调试**步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 以及后续迭代调试的全过程。

一、 设计成果(对设计流程的总结。需附上设计结构图, 可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF, 再插入到 LaTeX 中)

1 多周期 MIPS 处理器设计

本实验旨在设计并实现一个支持 MIPS 指令集子集的多周期处理器。与单周期 CPU 将所有操作压缩在一个时钟周期内完成不同, 我在本次设计中采用了多周期的硬件设计思想, 利用状态机将一条指令的执行过程拆分为取指、等待指令返回、译码、执行、访存、等待数据返回以及写回等若干阶段, 从而使不同类型指令能够按照自身需要占用不同数量的时钟周期。这样既避免了所有指令都受最长数据通路限制的问题, 也使 CPU 在面对访存等待等情况时能够通过状态机进行更加清晰和稳定的控制。

在本次多周期 CPU 的设计中, 整个状态机主要包括 INIT、IF、IW、ID、EX、LD、RDW、ST 和 WB 九个状态。其中 INIT 作为复位后的初始状态, 用于保证 CPU 从确定状态开始运行; IF 状态负责向指令存储器发出取指请求; IW 状态等待指令存储器返回有效指令; ID 状态完成指令译码和寄存器读数; EX 状态根据指令类型完成 ALU 运算、访存地址计算、分支条件判断或跳转地址生成; LD 和 RDW 状态用于 load 类指令的数据读请求与数据等待; ST 状态用于 store 类指令的数据写入; WB 状态则完成需要写寄存器指令的结果写回。

状态机的转移并不是简单地按照固定节拍推进, 而是结合外部握手信号进行控制。例如, 在 IF 状态下, 只有当 Inst_Req_Ready 有效时, CPU 才会进入 IW 状态; 在 IW 状态下, 只有当 Inst_Valid 有效时, CPU 才会进入 ID 状态。对于数据访存过程, load 指令需要先在 LD 状态等待 Mem_Req_Ready, 再进入 RDW 状态等待 Read_data_Valid; store 指令则在 ST 状态等待写请求被接受后回到 IF。通过这种设计, CPU 能够适应指令存储器和数据存储器可能存在的等待周期, 避免因为外部存储器尚未响应而提前改变内部状态。

PC 控制部分与状态机紧密配合。在本设计中, PC 不会在每一个时钟周期都更新, 而是在指令执行到合适阶段时才进行改变。普通指令按照顺序执行, 下一条指令地址为 PC+4; 分支指令则在 EX 阶段根据 isbranch 信号判断是否采用分支目标地址。分支目标地址由 PC_next 加上符号扩展并左移两位后的立即数偏移量得到, 从而

实现 MIPS 中相对于下一条指令地址的分支寻址方式。对于跳转类指令，PC 的来源又进一步分为两种情况。j、jal 等 J 型指令使用指令中的 26 位目标字段，与 PC_next 的高 4 位拼接后再左移两位，形成 32 位跳转地址；jr、jalr 等寄存器跳转指令则直接使用寄存器读出的值作为新的 PC。这样可以同时支持绝对跳转和寄存器间接跳转两类控制转移方式。对于 jal 和 jalr，由于还需要将返回地址写入寄存器，因此它们在完成 PC 更新后还会进入 WB 状态，将 PC_cur+8 写回到 31 号寄存器或指定目标寄存器中。为了保证 PC 相关信息在多周期执行过程中不丢失，我额外设置了 PC_cur 寄存器，在 IF 状态保存当前取到的指令地址。由于一条指令在后续可能经历多个周期，如果直接使用正在变化的 PC，可能会导致写回信息或 inst_retire 提交信息与实际指令不对应。因此，PC_cur 用来记录“当前正在执行的这条指令”的地址，并在写回或提交时与寄存器写使能、写回地址、写回数据一起输出，从而保证仿真验证时能够准确追踪每条指令的执行结果。

针对以上结构，我的多周期 CPU 结构如下：

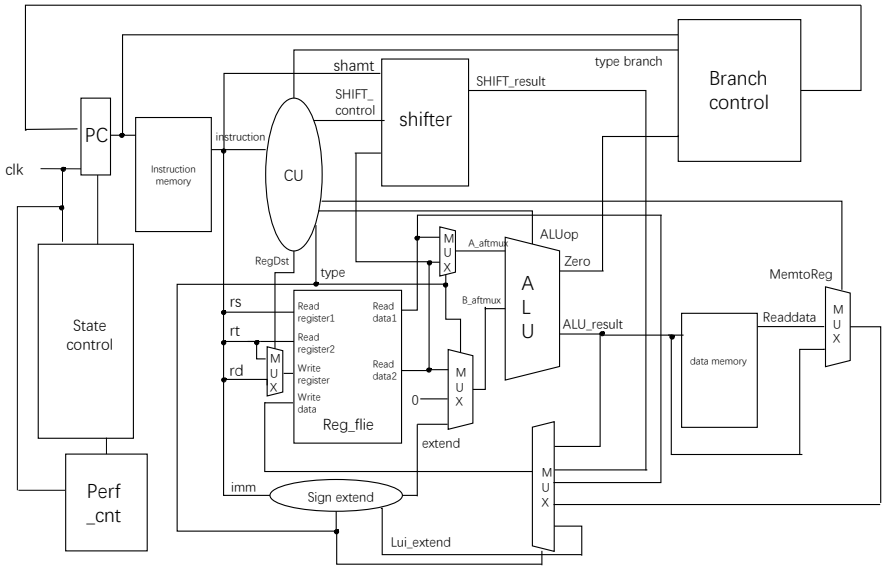


图 1: 多周期 CPU 状态机转换图

状态机设置与 PPT 一致，所以这里直接复用状态机：

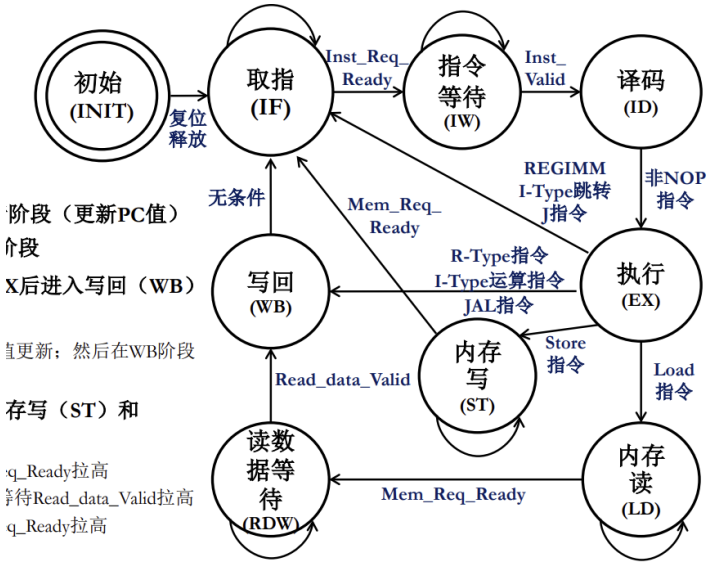


图 2: 多周期 CPU 状态机转换图

2 性能计数器设计

我还加入了简单的性能计数器模块，用于记录 CPU 的运行情况，方便后续仿真和性能分析。其中 cycle_cnt 用来统计 CPU 自复位结束后的总时钟周期数。每当时钟上升沿到来时，如果没有复位，该计数器就加一，并通过 cpu_perf_cnt_0 输出。这样可以直观观察程序运行一共消耗了多少个周期。另一个计数器是 inst_retired_cnt，用于统计已经完成提交的指令条数。由于本设计是多周期 CPU，不同指令结束的位置并不完全相同，所以不能简单地在固定状态下统一加一。对于需要写回寄存器的指令，我在 WB 状态认为其执行完成；对于 store 指令，由于不需要写回寄存器，因此在 ST 状态且 Mem_Req_Ready 有效时认为其提交完成；对于分支指令以及无链接跳转指令，则在 EX 状态完成 PC 更新后即可认为提交完成。基于这些条件，inst_retired_cnt 能够较准确地反映 CPU 实际完成的指令数量，并通过 cpu_perf_cnt_1 输出。通过周期计数和提交指令数两个指标，可以进一步计算程序运行时的平均 CPI，从而评价多周期 CPU 的执行效率。虽然本实验中的性能计数器实现较为基础，但它为后续分析不同指令类型的周期消耗、访存等待对性能的影响以及整体 CPU 运行效率提供了必要的数据支持。

与外设连接的部分较为简单，仅仅需要按照要求修改 puts 函数即可，这里不加叙述。

3 其他关键模块设计（如有）

与外设连接的部分较为简单，仅仅需要按照要求修改 puts 函数即可，这里不加叙述。

二、 开发成果（对代码开发流程的总结。需附上包含注释的 Verilog HDL 关键代码段、相应信号的仿真波形和信号变化的说明）

1 多周期 MIPS 处理器设计

由于大部分部件可以直接复用单周期 CPU 的部件，这里仅说明新加入部件的开发成果。

复位后状态机首先进入 INIT 状态，随后进入 IF 发起取指请求。当指令请求通道握手成功，即 Inst_Req_Ready 有效时，状态转入 IW 等待指令返回；当 Inst_Valid 有效时，指令被锁存到 valid_Instruction，状态进入 ID 进行译码。在 ID 阶段，如果读到的指令为 0，则直接返回 IF，否则进入 EX 执行阶段。EX 状态根据指令类型决定后续路径：加载指令进入 LD，存储指令进入 ST，分支和跳转类指令在 EX 完成后直接回到 IF，其余需要写回的指令进入 WB。这种设计将访存等待和指令返回等待都拆分为独立状态，能够适配外部总线的握手机制。

```
//part 1
always @ (posedge clk)
begin
    if(rst)
        current_state <= INIT;
    else
        current_state <= next_state;
end

//part 2

always @ (*)
begin
    case(current_state)
    INIT:begin
        next_state = IF;
    end

    IF:begin
        if(Inst_Req_Ready)
        next_state = IW;
        else
        next_state = IF;
    end

    IW:begin
        if(Inst_Valid)
        next_state = ID;
        else
        next_state = IW;
    end

    ID:begin
        if(valid_Instruction == 32'b0)
        next_state = IF;
        else
        next_state = EX;
    end
```

(a) 状态机代码 1

```
    EX:begin
        if(mode == `I_Load)
        next_state = LD;
        else if(mode == `I_Store)
        next_state = ST;
        else if((mode == `REGIMM)||(mode == `I_Branch)||(opcode == 6'b000010))
        next_state = IF;
        else
        next_state = WB;
    end

    LD:begin
        if(Mem_Req_Ready)
        next_state = RDW;
        else
        next_state = LD;
    end

    RDW:begin
        if(Read_data_Valid)
        next_state = WB;
        else
        next_state = RDW;
    end

    ST:begin
        if(Mem_Req_Ready)
        next_state = IF;
        else
        next_state = ST;
    end

    WB:begin
        next_state = IF;
    end

    default:next_state = INIT;

    endcase
end
```

(b) 状态机代码 2

图 3: 并排图片

```
//part 3
assign Inst_Req_Valid   = (current_state == IF);
assign Inst_Ready       = (current_state == IW)||(current_state == INIT);
assign Read_data_Ready      = (current_state == RDW)||(current_state == INIT);
assign MemRead          = (current_state == LD);
assign MemWrite         = current_state == ST;

always @(posedge clk)
    begin
        valid_Instruction <= (Inst_Ready && Inst_Valid)? Instruction: valid_Instruction;
    end
always @(posedge clk)
    begin
        valid_Read_data <= (Read_data_Ready && Read_data_Valid)? Read_data: valid_Read_data;
    end
```

图 4: 状态机代码 3

进行仿真后,状态机切换的波形图如下:

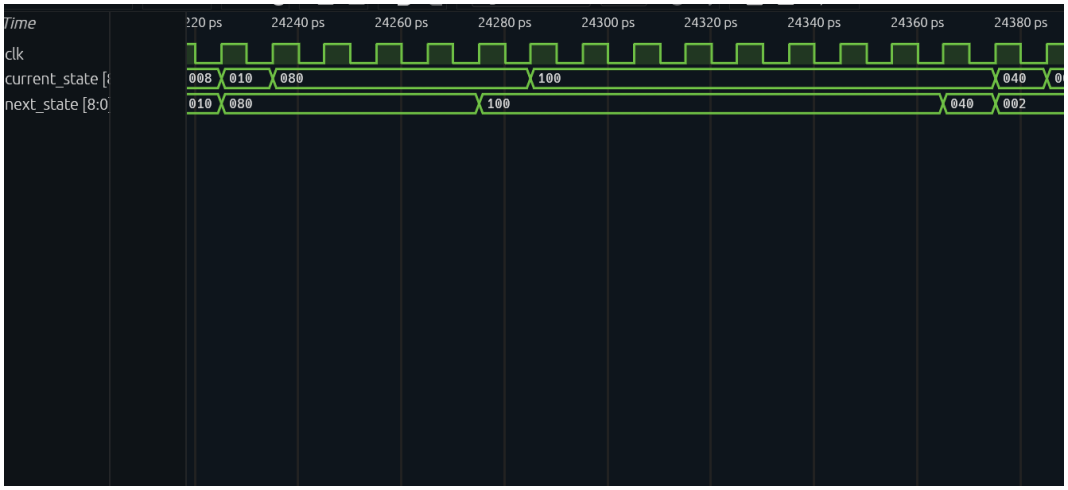


图 5: 状态机波形图

2 性能计数器设计

性能计数器的关键在于准确判断一条指令何时完成。对于普通运算类、移位类、跳转链接类、立即数运算类、加载类以及 lui 等需要写回寄存器的指令,CPU 在 WB 状态完成提交;对于存储指令,由于其不写回寄存器,因此在 ST 状态且 Mem_Req_Ready 有效时认为提交完成;对于条件分支、REGIMM 类分支以及普通跳转指令,由于这些指令在 EX 状态即可完成 PC 更新,因此在 EX 状态满足对应类型判断时进行计数。通过这种分类统计方式,可以保证不同执行路径的指令都能在真正完成时被计入提交数。

```
// performance counter
// The number of cycle
reg [31:0] cycle_cnt;
always @(posedge clk)
begin
    if(rst) cycle_cnt <= 32'd0;
    else cycle_cnt <= cycle_cnt + 32'd1;
end
assign cpu_perf_cnt_0 = cycle_cnt;

// The number of instructions retired
reg [31:0] inst_retired_cnt;
always @(posedge clk)
begin
        if(rst)
        inst_retired_cnt <= 32'b0;
        else
    begin
        if ((current_state == WB) ||
                (current_state == ST && Mem_Req_Ready) ||
                (current_state == EX && ((mode == `REGIMM) || (mode == `I_Branch) || (opcode == 6'b000010))))
                inst_retired_cnt <= inst_retired_cnt + 1'b1;
    end
end
assign cpu_perf_cnt_1 = inst_retired_cnt;
```

图 6: 性能计数器代码

```
[ssort] Suffix sort: * Passed.
Total Cycles : 52542794
Total Insts  : 683711
benchmark finished
time 1660.70ms
```

图 7: 性能计数器输出结果

3 其他关键模块设计(如有)

外设连接的 puts 函数如下:

```
int
puts(const char *s)
{
    //TODO: Add your driver code here
    unsigned int i = 0;
    while(s[i]){
        while ( *(uart + (UART_STATUS / 4)) & UART_TX_FIFO_FULL ) {
        }
        *(uart + (UART_TX_FIFO / 4)) = s[i];
        i++;
    }
    return i;
}
```

图 8: puts 函数代码

三、 学习总结(总结设计、开发与调试过程中遇到的问题,及实验中的收获与不足)

1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

比如 RTL 代码中出现的逻辑 bug、逻辑仿真和 FPGA 调试过程中的难点等。
出错模块指设计章节中列出的各个模块。
问题类别包括逻辑电路设计、RTL 语法、行为仿真和逻辑综合。
产生原因包括 xxx 概念理解不清、xxx 模块设计环节出现 xxx 问题、xxx 设计不符合 xxx 设计规范等。

表 1: 实验问题总结

序号	问题类别	出错模块	问题现象	产生原因	解决办法
1	逻辑电路设计	写使能控制信号	写使能持续拉高	写使能模块设计环节出现没有考虑状态机的问题	修改了写使能的定义,加上了当前状态的判断
2	RTL 语法	运算类别变量 `type`	系统将 `type` 当作系统函数报错	`type` 变量与系统函数撞名	把 `type` 改成了 `mode`
3	逻辑电路设计	寄存器写数据	写入结果小 4	我在这里按照多周期 CPU 处理,没有考虑延迟槽	把 PC+4 改回了 PC+8

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

通过本次多周期 CPU 实验,我对处理器设计方法有了更直观的理解。相比单周期 CPU,多周期 CPU 不再要求一条指令在一个时钟周期内完成,而是将取指、译码、执行、访存和写回拆分为多个阶段,并通过状态机控制各阶段的先后关系。这种设计方式虽然增加了控制逻辑复杂度,但有效降低了单周期组合路径过长带来的时序压力,也让我更清楚地认识到 CPU 中不同指令执行周期差异的来源。

本次实验中较大的亮点是状态机划分较为清晰,并且能够结合外部握手信号处理取指和访存等待问题;PC 更新逻辑也覆盖了顺序执行、条件分支、J 型跳转和寄存器跳转等多种情况。同时,通过性能计数器统计周期数和提交指令数,我进一步理解了 CPI、访存等待和控制转移对处理器运行效率的影响。

不足之处在于,部分控制信号仍然采用较长的组合表达式实现,虽然功能上能够通过仿真,但可读性和可维护性还有提升空间;此外,验证过程主要依赖波形观察和典型指令测试,对于边界情况和异常组合的覆盖还不够系统。后续如果继续完善该 CPU,我计划将译码控制信号进一步模块化,并补充更完整的指令级测试用例,尤其加强对分支跳转、访存对齐、握手等待和性能计数器边界行为的验证。

表 2: 收获总结

类别	名称	体会	后续计划
设计方法	多周期状态机设计	掌握了用状态机划分 CPU 执行阶段的方法，能够将取指、译码、执行、访存和写回分开控制。但在初期设计时，对不同指令执行路径的周期差异考虑不够充分。	后续在设计前先整理完整的数据通路和状态转移图，再根据指令类型逐项检查状态覆盖情况。
RTL 编写	控制信号与 PC 更新逻辑	能够使用组合逻辑实现分支判断、跳转地址生成和 PC 更新控制。但部分表达式较长，信号之间的依赖关系不够直观，后期调试成本较高。	后续将复杂控制逻辑拆分为更清晰的中间信号，并加强命名规范，提高代码可读性和可维护性。
验证方法	波形仿真与性能计数	通过观察状态机、PC、访存握手和性能计数器波形，能够定位指令执行过程中的控制问题。但测试用例仍偏向典型情况，边界覆盖不足。	后续补充分支跳转、load/store 等关键指令的专项测试，并建立更系统的自动化验证流程。

四、 扩展思考(对讲义中思考题(如有)的理解和回答)

• **思考题 1：上图中 volatile 关键字的作用是什么？如果去掉会出现什么后果？**。

volatile 关键字的核心作用是明确告知编译器该变量所指向的内存地址具有易变性与不可预测性，从而强制编译器对该指针的每一次读写操作都必须直接映射到实际的物理内存或外设寄存器地址，全面禁止编译器的过度优化。如果去除该关键字，高级编译器在开启代码优化(如常用于提升运行效率的 -O2 或 -O3 选项)后，其静态分析逻辑会误认为该地址上的值在软件生命周期内没有被当前软件逻辑改变，进而将重复的寄存器读取操作替换为直接访问 CPU 寄存器或缓存中的副本，或者将连续写入发送队列的操作精简合并为仅保留最后一次写入的无效动作。这种错误的优化行为将直接导致驱动程序无法实时捕获底层硬件的最新状态、向 UART 发送队列连续写入字符时发生数据严重丢失、或者程序陷入死循环检测状态标志的故障，使得软硬件之间的数据通路彻底失效。

五、 实验时长

在课后，你花费了大约 15 小时完成此次实验，其中各个阶段的用时如表 3所示。

步骤	时长(小时)	说明
设计	3	
开发	6	
调试	6	

表 3: 实验时长